18 de abril de 2026

Fundamentos de Sockets en Python

PROGRAMACION DE APLICACIONES DE RED

Luis Guillermo Mejenes Chacon y Javier Rodríguez Rodríguez
230300756 | 230300966

Fundamentos de Sockets en Python

1. Parte Teórica

a) Direccionamiento en Internet

El internet lo podemos ver como una ciudad con muchas casas. Para que dos dispositivos se comuniquen, necesitan saber “dónde” está la casa, como la región y "qué puerta" tocar, como el número de casa. Ya para entender esto de una manera más realista hay que hablar de direcciones IP, puertos, sockets y hasta protocolos.

- **Dirección IP (IPv4 e IPv6):** Es la identificación única de un dispositivo en una red.
    - **IPv4:** Usa 32 bits, pero usa una notación decimal separada por puntos. Se divide en una **parte de red** y una **parte de host** mediante la máscara de subred. Su gran limitación fue el agotamiento de direcciones (solo hay $2^{32} \approx 4.3$ mil millones). (ej. 192.168.1.1).
    - **IPv6:** Usa 128 bits y se escribe en notación hexadecimal y ofrece un número casi infinito de direcciones. Diseñado no solo por espacio, sino por eficiencia. Elimina la necesidad de NAT (Network Address Translation) y permite que cada dispositivo en el planeta tenga su propia IP pública global. Las direcciones se abrevian eliminando ceros a la izquierda y usando (::) para grupos de ceros consecutivos. (ej. 2806:10be:7:c6ab:f94f:8f72:2d8f:2bdf).
- **Número de puerto:** Imagina que la IP es la dirección de un edificio; el puerto es el número del departamento. Permite que múltiples aplicaciones (navegador, juegos, apps) usen la red al mismo tiempo sin mezclarse. Van del **0 al 65535**.

  Funciona en la **Capa de Transporte** (Capa 4 del modelo OSI).

    - **Puertos Bien Conocidos (0-1023):** Reservados para servicios estándar (ej. HTTP: 80, HTTPS: 443, SSH: 22).
    - **Puertos Registrados (1024-49151):** Usados por aplicaciones específicas (ej. bases de datos como MySQL: 3306).
    - **Puertos Dinámicos/Privados (49152-65535):** Asignados temporalmente por el sistema operativo cuando tu cliente abre una conexión.

- **Concepto de Socket:** Es el punto final de una comunicación. Técnicamente, es la combinación de una **Dirección IP + un Número de puerto** (ej. 192.168.0.10:80). Es el "enchufe" virtual donde se conecta el flujo de datos.
- **Direcciones Especiales:**
    - **localhost / 127.0.0.1:** Es la dirección de "bucle de retorno" (loopback). Se usa para que una computadora se hable a sí misma. Útil para pruebas.
    - **0.0.0.0:** En servidores, significa "escuchar en todas las interfaces de red disponibles". En otros contextos, representa una red desconocida o no configurada.

b) Protocolos de Transporte

Son las reglas que deciden cómo viajan los paquetes de datos.

- **TCP (Transmission Control Protocol):** Es el protocolo formal y orientado a conexión. Antes de enviar datos, establece una conexión (handshake). Si un paquete se pierde, lo vuelve a pedir. Garantiza que los datos lleguen completos y en orden.
    - *Usos:* Navegación web (HTTP), correo electrónico, transferencia de archivos (FTP).
- **UDP (User Datagram Protocol):** Es el protocolo "veloz" pero informal. No establece conexión previa ni verifica si el mensaje llegó. Envía los datos y espera que el destino los reciba. Es un protocolo sin estado. No hay confirmación de recepción. Cada paquete (llamado datagrama) es independiente y es ideal para aplicaciones de tiempo real donde perder un paquete es preferible a esperar a que se reenvíe.
    - *Usos:* Streaming de video, llamadas VoIP, videojuegos en línea donde la velocidad importa más que un pequeño error.

c) Funciones de Sockets en Python

En Python, usamos el módulo socket. La descripción de las funciones es:

Para el Servidor:

- socket(): Crea un nuevo objeto de socket.
- bind(): Asocia el socket a una dirección IP y un puerto específico.
- listen(): Pone al servidor en modo de espera, listo para recibir clientes.

- accept(): Bloquea la ejecución hasta que llega un cliente; devuelve un nuevo socket para la comunicación y la dirección del cliente.

Para el Cliente:

- connect(): Inicia la conexión hacia el servidor (específicamente para TCP).

Intercambio de Datos:

- send() / recv(): Se usan en TCP para enviar y recibir datos en una conexión establecida.
- sendto() / recvfrom(): Se usan en UDP. Como no hay conexión fija, en cada envío se debe especificar el destino.

Finalización:

- close(): Cierra el socket y libera los recursos del sistema.

d) Cuadro comparativo Comparar TCP vs UDP

Característica	TCP	UDP
Conexión	Orientado a conexión (establece enlace).	No orientado a conexión (envío directo).
Confiabilidad	Alta (retransmite datos perdidos).	Baja (puede haber pérdida de datos).
Velocidad	Más lento (debido a las comprobaciones).	Muy rápido (sin sobrecarga de control).
Orden	Garantiza que los datos lleguen en orden.	Los datos pueden llegar en cualquier orden.
Usó común	Web, Email, SSH, Transferencia de archivos.	Videojuegos, Streaming, DNS, Videollamadas.

2. Parte práctica

a) Implementación TCP

Servidor:

• Crear socket TCP

• Escuchar conexiones

• Recibir mensaje

• Enviar respuesta

• Cerrar conexión

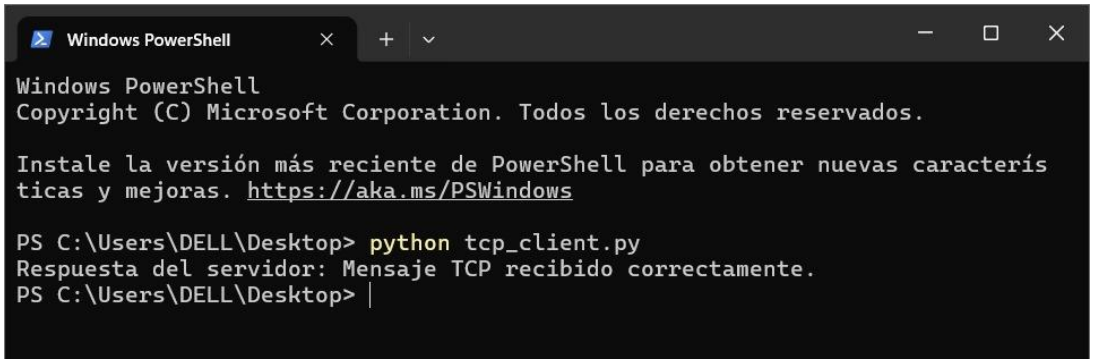
Cliente:

• Conectarse al servidor

• Enviar mensaje

• Recibir respuesta

• Mostrar resultado



b) Implementación UDP

Servidor:

• Crear socket UDP

• Recibir mensaje

• Responder al cliente

Cliente:

• Enviar mensaje sin conexión

• Recibir respuesta

c) Pruebas

• Ejecutar servidor y cliente

• Verificar comunicación

• Capturar evidencias

3. Análisis

Responder las siguientes preguntas basándose en la experiencia de

implementación:

• Diferencias entre TCP y UDP en la práctica

Al implementar ambos en Python, la diferencia más notable es la gestión de la conexión:

- **En TCP:** Se siente como una llamada de celular. El código de cliente tuvo que esperar a que el servidor aceptara antes de poder enviar un solo byte. Además, tuvimos que manejar el flujo de datos como un stream; si enviaba un mensaje muy largo, a veces el debía llamarse varias veces para recomponerlo.
- **En UDP:** Es como enviar una postal. El código es más simple. Simplemente se uso sendto() disparando los datos hacia una IP y puerto. La gran diferencia práctica es que, si enviaba datos y el receptor no estaba listo, Python no lanzaba ningún error de inmediato; los datos simplemente se perdían.

• ¿Qué ocurre si el servidor no está activo?

- **En TCP:** El script del cliente falla inmediatamente. Esto sucede porque el sistema operativo intenta el handshake y, al no recibir respuesta del puerto, corta el proceso.
- **En UDP:** Curiosamente, el cliente no sabe que el servidor está caído al momento de enviar. El método sendto() se ejecuta correctamente. Sin embargo, en algunas configuraciones locales, el sistema puede recibir un mensaje de puerto inalcanzable pero el envío inicial parece exitoso, aunque nadie reciba nada.

• ¿Qué pasa si el puerto es incorrecto?

Si el puerto está cerrado o es el equivocado:

- **Si el puerto está cerrado:** El comportamiento es idéntico al de un servidor inactivo (Error de conexión rechazada).
- **Si el puerto está abierto por otra aplicación:** El sistema operativo permitirá que el cliente se conecte (en TCP), pero la aplicación que escucha en ese puerto no entenderá los datos que le estamos enviando. Probablemente el servidor nos ignore o nos cierre la conexión al recibir datos que no siguen su protocolo, lo que en Python veríamos como una recepción vacía o un error de protocolo.

• ¿Se pueden conectar varios clientes al mismo tiempo?

La respuesta corta es **sí**, pero depende de cómo hayamos programado el servidor:

- **En TCP:** El servidor necesita usar **hilos** o **asincronía**. Aunque el socket de escucha puede recibir a varios, la función accept() solo atiende a uno a la vez. Si no usamos hilos, el segundo cliente se quedará bloqueado en la cola hasta que el servidor termine de hablar con el primero y vuelva a ejecutar accept().
- **En UDP:** Sí, y de forma mucho más natural. Como no hay una "conexión" permanente, el servidor simplemente recibe paquetes de quien sea. El método recvfrom() nos da los datos y la dirección de quien los envió, permitiéndonos responder a múltiples clientes de forma intercalada sin necesidad de hilos complejos para mantener la sesión.

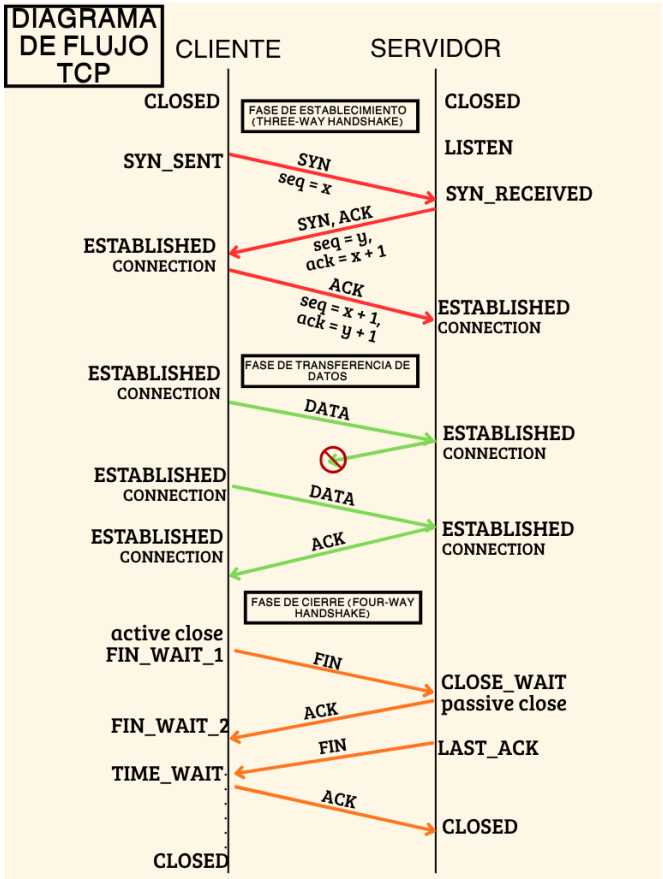
4. Diagrama

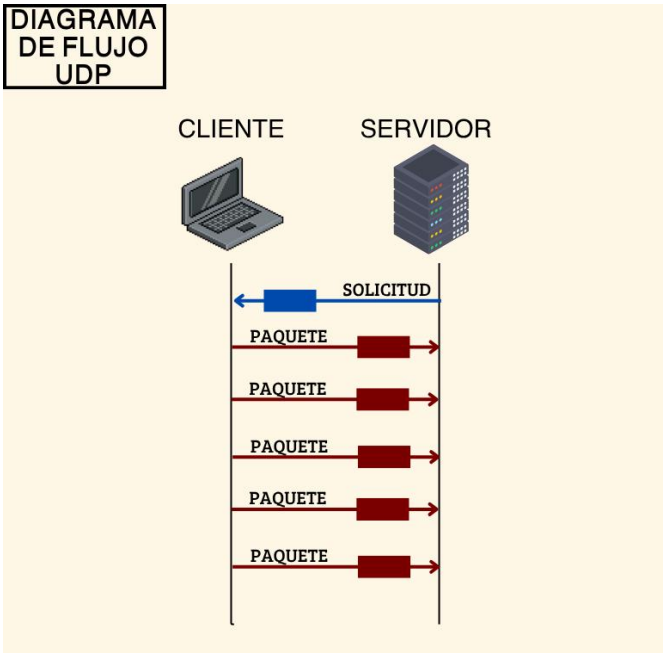
Elaborar un diagrama que represente:

• El flujo completo de comunicación TCP

• El flujo completo de comunicación UDP

• Señalar claramente las diferencias entre ambos





5. Conclusiones

Tras realizar la investigación teórica y la implementación práctica de sockets en Python se concluye que la elección entre TCP y UDP no depende de cuál es mejor sino de las necesidades específicas de la aplicación ya que mientras TCP es indispensable para servicios que requieren integridad absoluta y orden en los datos como la transferencia de archivos o la navegación web UDP es la opción superior para servicios de tiempo real como streaming o videojuegos donde la velocidad es más crítica que la pérdida ocasional de paquetes.

Además, se demostró que la confiabilidad de TCP tiene un costo en complejidad debido al proceso de establecimiento de conexión o handshake y la necesidad de confirmaciones constantes mediante ACKs mientras que UDP ofrece una eficiencia máxima con un código más simple al ser un protocolo sin estado que no verifica si el mensaje llegó a su destino.

Por otro lado se observó que la gestión de múltiples clientes varía drásticamente pues en TCP el servidor requiere el uso de hilos o asincronía para no bloquear la atención a nuevos usuarios mientras atiende una conexión a diferencia de UDP donde la naturaleza sin conexión permite recibir paquetes de múltiples orígenes de forma más natural e intercalada finalmente se comprendió que el éxito de la comunicación depende de la correcta configuración del socket mediante la IP y el puerto resaltando que un error en estos parámetros provoca fallos inmediatos de conexión en TCP mientras que en UDP los datos pueden enviarse sin que el emisor note que el receptor no está activo o disponible.

Repositorio de Git Hub:

https://github.com/JaceBeleren11/Kuraiantos-b-